YOOBEE
COLLEGE OF CREATIVE INNOVATION

Master of Software Engineering — Level 8

MSE802 Quantum Computing

Assessment 2 — Quantum Project

Individual Reflection Report

Learning entanglement and quantum machine learning by measuring them

Student	Sandhya Tiwari
Student ID	270918511
Role in the group	Task 1 (Bell state in Cirq), Task 4 (quantum machine-learning analysis), reference checking and proofreading
Group partner	Anish Shrestha (270889991), team lead
Lecturer	Dr Helena Bahrami
Reflective model	What? So what? Now what? (Rolfe et al., 2001)
Date	4 November 2026
Referencing style	APA 7th edition

Auckland, New Zealand

Declaration

I, Sandhya Tiwari, declare that this reflection is my own individual work for MSE802 Assessment 2. It describes my own experience of the group project with Anish Shrestha. Sources are acknowledged in APA 7th style, and the project facts and numbers come from our submitted notebooks, results files and Git history.

Use of digital tools: a generative AI assistant was used for language editing and for drafting parts of the project code and documents; the experiences, judgements and plans in this reflection are my own. [Edit this sentence to match Yoobee's AI-use policy and your actual use.]

Name	Student ID
Sandhya Tiwari	270918511

Project Links and How to Run Online

Every result in this document can be reproduced online, with nothing to install. Copy a link below and paste it into the address bar of a web browser.

GitHub repository (all code, results and reports):
https://github.com/anishnz/quantumcomputing_YB_Anish

Google Colab – run ALL tasks (recommended):
https://colab.research.google.com/github/anishnz/quantumcomputing_YB_Anish/blob/main/Assignment_2_Quantum/Colab_Run_All_Tasks.ipynb

Google Colab – Task 1 (Bell state, Cirq):
https://colab.research.google.com/github/anishnz/quantumcomputing_YB_Anish/blob/main/Assignment_2_Quantum/Task_1_Bell_State/Task1_Bell_State_Cirq_Quokka.ipynb

Google Colab – Task 2 (Qiskit circuit + QASM):
https://colab.research.google.com/github/anishnz/quantumcomputing_YB_Anish/blob/main/Assignment_2_Quantum/Task_2_Qiskit_QASM/Task2_Qiskit_Circuits_and_QASM.ipynb

Google Colab – Task 3 (quantum tic-tac-toe):
https://colab.research.google.com/github/anishnz/quantumcomputing_YB_Anish/blob/main/Assignment_2_Quantum/Task_3_Quantum_TicTacToe/Task3_Quantum_TicTacToe_Completed.ipynb

Google Colab – Task 4 (quantum machine learning):
https://colab.research.google.com/github/anishnz/quantumcomputing_YB_Anish/blob/main/Assignment_2_Quantum/Task_4_Quantum_ML/Task4_Quantum_ML_Modified.ipynb

Download the whole project as a ZIP:
https://github.com/anishnz/quantumcomputing_YB_Anish/archive/refs/heads/main.zip

How to run the project online in Google Colab (step by step)

1. Copy the link "Google Colab – run ALL tasks" above and paste it into the address bar of Chrome, Edge or Safari, then press Enter.
2. Sign in with any Google account if Colab asks you to (Colab is free).
3. Click Runtime → Run all. If Colab shows "Warning: This notebook was not authored by Google", click Run anyway.
4. Wait about 15–20 minutes. The notebook downloads the GitHub repository, installs Cirq and Qiskit, checks which Quokka devices are online, runs Tasks 1–4 with 1000 shots (Task 4 sends 900 circuits to Quokka), shows every table and chart, runs the laptop-vs-Quokka comparison and the alternative code.
5. Read the results under each step. Every result says where it ran: QUOKKA DEVICE (with the device name, e.g. quokka2) or LOCAL SIMULATOR if no Quokka device is online.
6. To keep your own copy, click File → Save a copy in Drive. Step 7 of the notebook can also download all outputs as a ZIP.
7. To run only one task, open that task's Colab link above, then click Runtime → Run all. Its first code cell downloads the project automatically.
8. To run on your own computer instead: download the ZIP link above, unzip it, open the Assignment_2_Quantum folder and double-click run_project.bat (Windows) or run_project.command (macOS). Python 3.12 or newer is needed.

Table of Contents

Declaration..... 1
Project Links and How to Run Online..... 2
Table of Contents..... 3
Summary..... 4
1. Introduction..... 5
1.1 Why reflect at all?..... 5
1.2 How does the framework work?..... 5
1.3 What was my share of the project?..... 5
2. Cycle 1 – The Bell State (Task 1)..... 7
2.1 What? What did I do and see?..... 7
2.2 So what? What does it mean to me?..... 7
2.3 Now what? What will I do with this?..... 8
3. Cycle 2 – The Quantum Machine-Learning Classifier (Task 4)..... 9
3.1 What? What did I do and see?..... 9
3.2 So what? What does it mean to me?..... 10
3.3 Now what? What will I do with this?..... 11
4. Cycle 3 – Working in a Two-Person Team..... 12
4.1 What? What happened in our teamwork?..... 12
4.2 So what? What does it mean to me?..... 12
4.3 Now what? What will I do with this?..... 12
5. What I Learned Overall..... 13
6. Conclusion..... 13
References..... 14
Appendix – Evidence Behind This Reflection..... 15

(If page numbers are missing, right-click the table and choose Update Field.)

Summary

In this reflection I look at my part of our group's quantum project: building the **Bell state** for Task 1 and analysing the **quantum machine-learning classifier** for Task 4. I also checked references and proofread the group report. I use the **"What? So what? Now what?"** framework of Rolfe et al. (2001), which grew out of Borton's (1970) three questions. I chose it because it is short and practical, and it keeps asking me what an experience *means* rather than only what happened. My biggest learning was that numbers taught me more than definitions did. Seeing 501 and 499 in a 1000-shot Bell run, and seeing a 100-iteration training run slow down because every step needs nine circuits, made the lecture ideas real. My main weakness was that I waited too long before asking questions about code I did not understand.

1. Introduction

1.1 Why reflect at all?

For me, this project was the first time I wrote and ran quantum programs that were marked. A good reflection should help me learn from that first time. Moon (2004) describes reflection as a way to *process* learning that is otherwise left half-finished, and Hattie and Timperley (2007) show that feedback helps most when it answers three questions: where am I going, how am I going, and where to next. The Rolfe framework asks almost the same three questions, which is another reason I picked it.

1.2 How does the framework work?

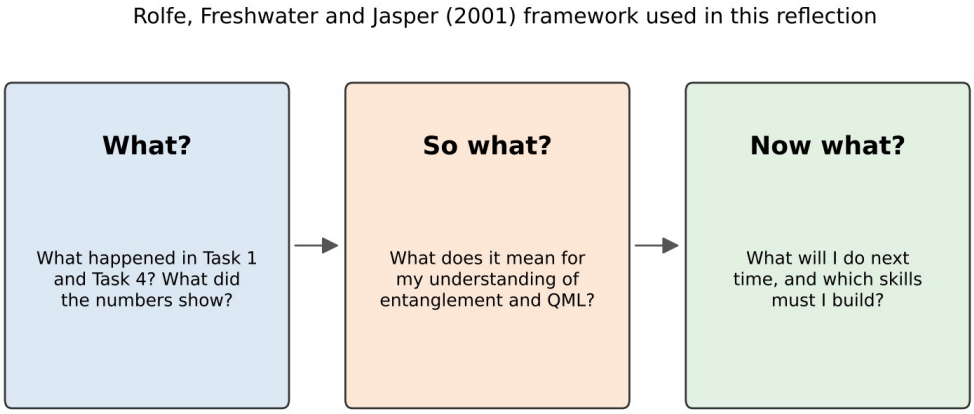


Figure 1. *The three questions of the Rolfe et al. (2001) framework, as used in this reflection*

What? describes the situation: what I did, what I saw, what the results were. **So what?** asks what this means: what I learned about the theory, about myself and about working in a team. **Now what?** turns the meaning into action: what I will do in the next course and in my career. I go through the cycle three times, once for each important experience: the Bell state, the machine-learning task and our teamwork.

1.3 What was my share of the project?

Table 1. *My responsibilities in the group project*

Part of the project	My role	Partner's role
Task 1 – Bell state in Cirq, theory, statistics	built and analysed it	reviewed; wrote the shared Quokka helper
Task 4 – circuit analysis, data input, quantum aspects	analysed and wrote it	reviewed
Task 4 – iteration log, two required plots, local versions	built and tested them	integrated into the runner
Reference list (APA 7) and DOI checking	checked every source	added new sources
Proofreading of the group report	read every section	final editing as team lead
Tasks 2 and 3, runner, GitHub, Colab, launchers	tested and gave feedback	built them (team lead)

Anish, our team lead, planned the project and built the shared parts. I owned two complete tasks from start to finish, so I could show my own understanding while still depending on the common code. This arrangement suited me: I had a clear target and someone to ask when the environment caused trouble.

2. Cycle 1 – The Bell State (Task 1)

2.1 What? What did I do and see?

Task 1 asked for the entangled state (|00⟩ + |11⟩)/√2 in Cirq, run on the Quokka device. I created two `LineQubit` objects, applied a Hadamard gate to the first qubit and a CNOT gate from the first to the second, and measured both. Before sending anything to the device, I worked out the expected state by hand with matrices, as we practised in the lectures, and checked it against Cirq's exact simulation: amplitude 0.7071 for |00⟩ and |11⟩, and zero for the others.

The first results in our project came from a local simulator, because the Quokka address was offline for several days. When four Quokka devices came back on 4 November 2026, our run on quokka2 returned **501 × 00 and 499 × 11**, with no 01 or 10 at all. Quokka itself needed only about 3.1 ms of processing for all 1000 shots; the rest of the 0.6338 s was the internet round trip.

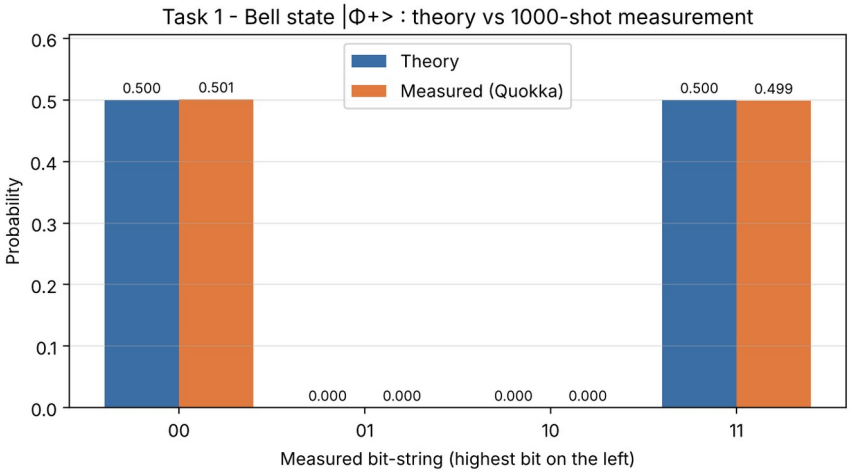


Figure 2. *My Task 1 result on Quokka: theory and measured probability for each outcome*

2.2 So what? What does it mean to me?

The first thing I noticed was something I nearly reported as a mistake. In an earlier simulator run the split was 508/492, not 500/500, and I wondered if the circuit was slightly wrong. To check my worry, I repeated the 1000-shot Bell experiment **300 times** on my laptop simulator and plotted how the 00 count varies (Figure 3).

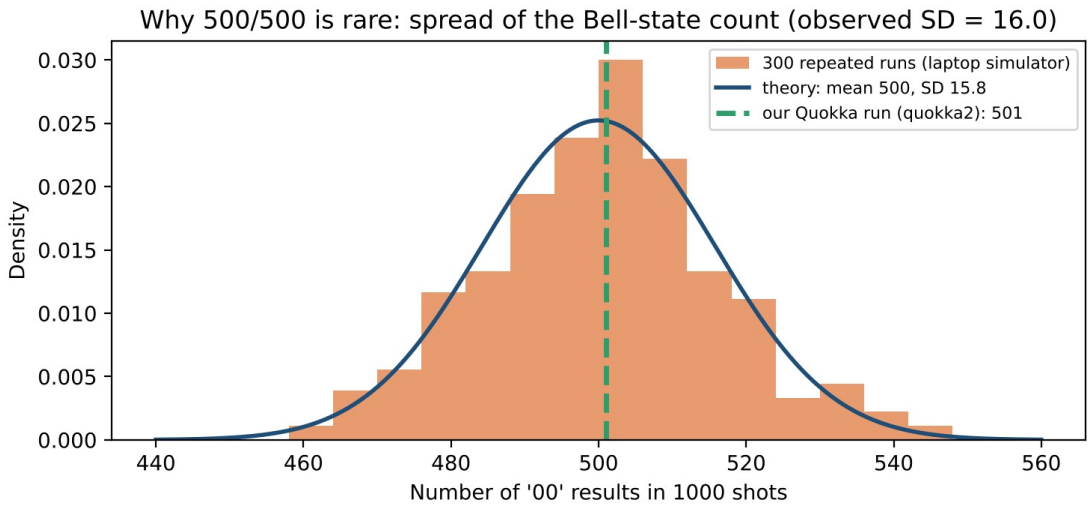


Figure 3. *Spread of the '00' count over 300 repeated 1000-shot Bell runs, with the binomial theory and our Quokka result*

The counts ranged from 459 to 545. Their standard deviation was 16.0, very close to the theoretical $\sqrt{(1000 \times 0.5 \times 0.5)} \approx 15.8$. Only 2.3% of runs gave exactly 500, while 94% stayed within two standard deviations. So a perfect 500/500 would actually be *unusual*. This changed how I read every result in the project. A measured probability is an estimate with an error bar, not a fixed value. I now look at the **absence of 01 and 10** as the real proof of entanglement, because ideal theory says those outcomes have probability zero. The exact split between 00 and 11 is just a coin-flip statistic.

The second meaning was about entanglement itself. Each qubit alone behaves like a fair coin, yet the pair always agrees. In the lectures this sounded abstract. When I wrote the state as a product $(a|0\rangle + b|1\rangle) \otimes (c|0\rangle + d|1\rangle)$ and found that no values of a, b, c and d can give $(|00\rangle + |11\rangle)/\sqrt{2}$, I finally understood why the textbook calls it *not separable* (Nielsen & Chuang, 2010).

2.3 Now what? What will I do with this?

- Always report a shot-noise range (for example 500 ± 32 for two standard deviations) next to any measured count, so readers can judge whether a difference matters.
- Prove a property with the outcome that theory forbids (here 01 and 10), not only with the outcomes that are allowed.
- Do the hand calculation first. It took ten minutes and later saved me from treating normal randomness as a bug.

3. Cycle 2 – The Quantum Machine-Learning Classifier (Task 4)

3.1 What? What did I do and see?

Task 4 gave us a notebook that creates tiny 2 × 2 black-and-white images (bars and stripes) and trains a quantum circuit to sort them into two classes. My jobs were to analyse the circuit and the problem, find which part of the code is the input to the circuit, explain the quantum part, record the output of every training iteration, add two plots (iteration vs output and iteration vs time), and make a version that runs without the quantum circuit.

The input turned out to be very direct. Each pixel becomes one qubit, and a black pixel is written as an X gate (**basis encoding**). The classifier is a small **tree tensor network**: pairs of qubits are combined by trainable rotation gates, and the result is read from one qubit, similar to the hierarchical classifiers of Grant et al. (2018). The optimiser (SPSA) only needs two circuit evaluations per step (Spall, 1992), but our monitoring added one more, and each evaluation runs one circuit per training image.

I added a timer and a record of the loss inside the optimiser loop. The run that the final report uses was executed on the Quokka device (quokka2): **900 circuits** in **606.2 s**, about **4.049 s per iteration**. The loss reached 0.0 and all four images were classified correctly.

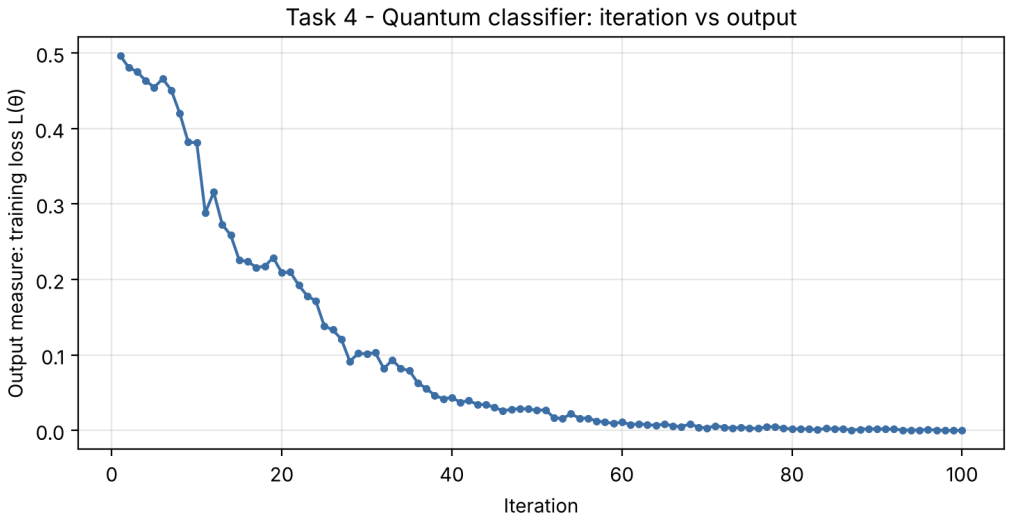


Figure 4. *Iteration versus output measure (loss) for the quantum classifier and the local versions*

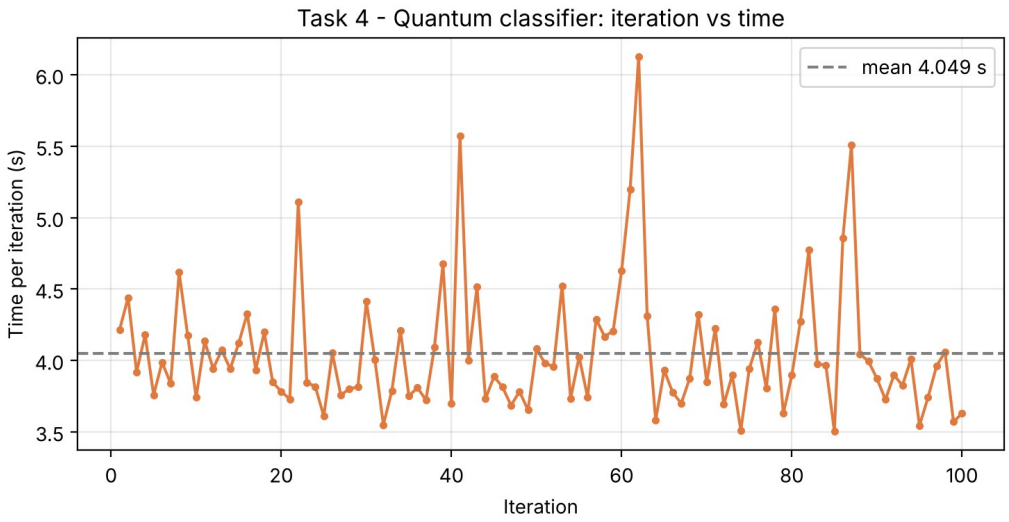


Figure 5. *Iteration versus time taken: every quantum iteration waits for several circuit runs*

For the local version I made three variants. **Local-A** simulates the same circuit exactly with NumPy matrices. **Local-B** is an ordinary logistic regression. **Local-C** is logistic regression with extra pairwise pixel features. Table 2 shows the outcome.

Table 2. *Efficiency and effectiveness of the quantum run and my three local versions (from task4_comparison.csv)*

Version	Time per iteration (s)	Final loss	Images correct (of 4)
Quantum circuit (Quokka)	4.04870	0.0	4
Local-A: same circuit, exact NumPy	0.00116	0.0003	4
Local-B: classical logistic regression	0.00005	0.2961	3
Local-C: classical + pairwise features	0.00019	0.2057	3

3.2 So what? What does it mean to me?

Two results surprised me. The first was **speed**. The exact local simulation of the same circuit was about **3490 times faster per iteration** than the shot-based quantum run, and it reached the same answer. For a four-qubit problem, a quantum computer adds waiting time (network, queue, shots) without adding power, because a laptop can store all 16 amplitudes easily. Biamonte et al. (2017) and Schuld and Petruccione (2021) both warn that quantum machine learning only makes sense where the classical simulation becomes too large. Our project is a teaching example, not a speed contest, and I now understand why.

The second surprise was **effectiveness**. The simple classical models could not classify all four images: they reached only three of four, even though one of them had extra features. The reason is that bars versus stripes for these images behaves like an XOR problem, which no single straight line can separate. The tree circuit combines pairs of pixels through entangling gates, so it can represent that kind of pattern. This was my clearest lesson: the quantum part was not faster, but its *structure* matched the problem. A classical model with the right structure (Local-A copies the circuit's maths) does just as well.

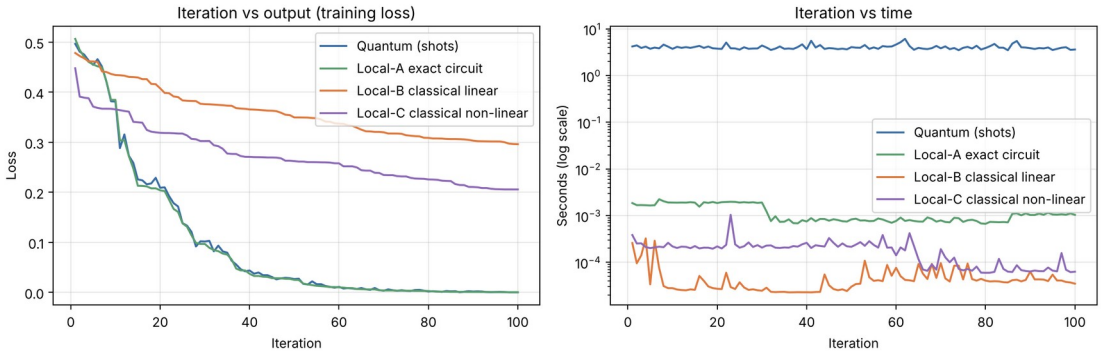


Figure 6. *Comparison of the quantum run and the local versions (loss and time)*

I also learned something about myself as an analyst. My first draft described SPSA in detail, but the brief said to explain the *quantum* aspects, not the optimiser. Anish pointed this out when he reviewed my section. At first I felt a bit discouraged, because I had worked hard on that part. Then I saw that he was right: the marker wants to see which data enters the circuit and what each gate does. Cutting my favourite paragraph made the section stronger.

3.3 Now what? What will I do with this?

- Before analysing a new algorithm, write down the question "what would a classical method need to solve this?". The XOR insight came only after I asked it.
- Measure time per step from the start, not only the total. The per-iteration plot made the cost of shots and network calls visible.
- Read the brief's verbs carefully (analyse, identify, explain) and plan one paragraph for each of them, so effort goes where marks are.

4. Cycle 3 – Working in a Two-Person Team

4.1 What? What happened in our teamwork?

We worked through short meetings, chat messages and our shared GitHub repository. Anish set the overall plan, and we agreed on shared rules early: 1000 shots for every circuit, results always saved with their source, and comments on every line of code. When the Quokka device was offline, he built a fallback so that my notebooks still produced labelled results. When the device came back, he changed the shared code to try several devices, and I re-ran my tasks on the real device without editing my own code.

4.2 So what? What does it mean to me?

I realised that depending on a teammate's code is a form of trust, and that the trust worked because his helper was well documented and easy to call with one line. I also noticed my own habit: when something in the shared code confused me, for example why a seed was needed or how bit strings were ordered, I first tried to solve it alone for too long. Cooperative-learning research says that groups learn best when members *explain their reasoning to each other* rather than only dividing work (Johnson & Johnson, 2009). The times I asked early, the answer took five minutes. The times I waited, I lost an evening.

Our different reflective models also show a difference in our roles. As team lead, Anish had to judge decisions that affected the whole project. My reflection is about learning the content deeply and about my own habits as a learner. Both views are needed in a team.

4.3 Now what? What will I do with this?

- Use a 20-minute rule: if I am stuck for 20 minutes on shared code, I ask.
- Offer to review the team lead's integration work, so knowledge is shared in both directions.
- Keep a short personal log of what I tried; it makes questions faster and reflections easier.

5. What I Learned Overall

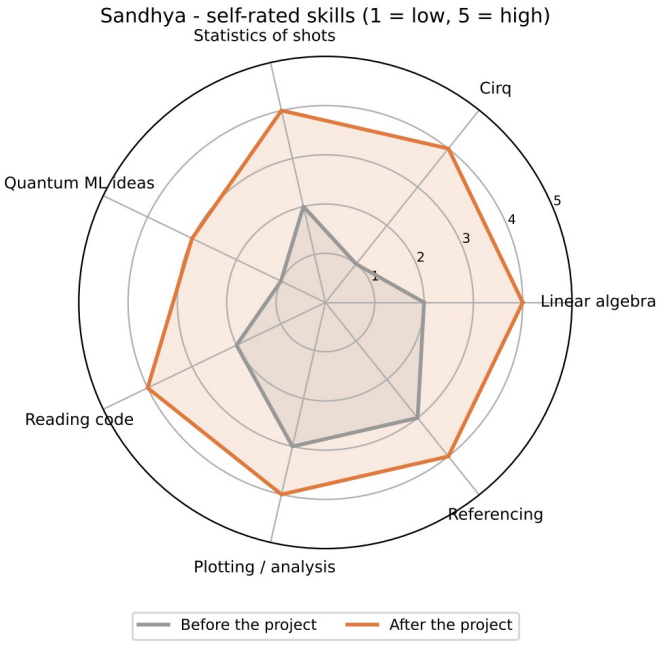


Figure 7. *Self-assessment: how seven of my skills changed during the project (scale 1 to 5)*

Figure 7 is my honest self-assessment. The biggest change was in **Cirq** and in the **statistics of shots**: I can now predict how much a count should vary and why. My understanding of **quantum machine learning** improved but is still the area where I feel least confident; I can explain our small classifier, but I would need more study for larger models. My referencing improved through checking every DOI in our list, a slow job that taught me how to recognise a reliable source.

Table 3. *My personal development plan*

Area	Next step	When
Quantum ML	Work through one chapter of Schuld and Petruccione (2021) and rebuild our classifier with 3 × 3 images	Before the next semester
Noise	Run the Bell state on a noisy simulator and compare with the ideal Quokka result	Within one month
Asking for help	Apply the 20-minute rule in my next group assignment	Next assignment
Communication	Present our Task 4 findings to classmates in five minutes, without equations	Next class presentation

6. Conclusion

The Rolfe questions helped me move from "I ran the code and it worked" to "I know what the numbers mean and what I will change". Task 1 taught me to read results with their natural randomness in mind. Task 4 taught me that a quantum model's value lies in its structure, not in raw speed, at least at this small size. Teamwork taught me to ask earlier and to trust well-documented shared code. I finish the project more confident in quantum programming, and with a clear list of what to do next.

References

Biamonte, J., Wittek, P., Pancotti, N., Rebentrost, P., Wiebe, N., & Lloyd, S. (2017). Quantum machine learning. *Nature*, *549*(7671), 195–202. https://doi.org/10.1038/nature23474

Borton, T. (1970). *Reach, touch, and teach: Student concerns and process education*. McGraw-Hill.

Grant, E., Benedetti, M., Cao, S., Hallam, A., Lockhart, J., Stojevic, V., Green, A. G., & Severini, S. (2018). Hierarchical quantum classifiers. *npj Quantum Information*, *4*, Article 65. https://doi.org/10.1038/s41534-018-0116-9

Hattie, J., & Timperley, H. (2007). The power of feedback. *Review of Educational Research*, *77*(1), 81–112. https://doi.org/10.3102/003465430298487

Johnson, D. W., & Johnson, R. T. (2009). An educational psychology success story: Social interdependence theory and cooperative learning. *Educational Researcher*, *38*(5), 365–379. https://doi.org/10.3102/0013189X09339057

Moon, J. A. (2004). *A handbook of reflective and experiential learning: Theory and practice*. RoutledgeFalmer.

Nielsen, M. A., & Chuang, I. L. (2010). *Quantum computation and quantum information* (10th anniversary ed.). Cambridge University Press. https://doi.org/10.1017/CBO9780511976667

Rolfe, G., Freshwater, D., & Jasper, M. (2001). *Critical reflection for nursing and the helping professions: A user's guide*. Palgrave Macmillan.

Schuld, M., & Petruccione, F. (2021). *Machine learning with quantum computers* (2nd ed.). Springer. https://doi.org/10.1007/978-3-030-83098-4

Spall, J. C. (1992). Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE Transactions on Automatic Control*, *37*(3), 332–341. https://doi.org/10.1109/9.119632

Appendix – Evidence Behind This Reflection

- Task 1 notebook and results: `Task_1_Bell_State/` (counts JSON with source and device).
- Task 4 modified notebook and logs: `Task_4_Quantum_ML/results/task4_quantum_iterations.csv` and the local versions' CSV files.
- Shot-noise experiment (300 × 1000 shots): `Report/Reflections/figures/S_shot_noise_stats.json`.
- Group repository: https://github.com/anishnz/quantumcomputing_YB_Anish.